

CardioCare: 심장병 예측 종단간 ML 시스템

CardioCare는 심장병 위험 신호를 추정하는 교육용 종단간 머신러닝 시스템이다. 이 프로젝트의 기본 원칙은 "알리되, 결정하지 않는다"이다. 모델은 환자 record가 심장병 위험과 얼마나 비슷한지 알려 주지만, 진단이나 치료 결정을 자동으로 내리는 시스템이 아니다. 결과는 심장 전문가가 차트를 검토할 때 참고하는 보조 정보로만 사용되어야 한다.

데이터는 UCI Heart Disease의 네 개 processed 파일(Cleveland, Hungarian, Switzerland, VA)을 통합해 사용했다. 원래 target은 여러 질병 정도를 나타내는 다중 클래스이지만, 이 과제에서는 target=0을 정상, target=1을 심장병 관측 기록으로 이진화했다. repository에는 정리된 CSV가 포함되어 있고, 파일이 없을 경우 preprocessing 모듈이 UCI 원본 파일에서 결정론적으로 다시 생성할 수 있다.

재현성은 구현의 일부로 다뤘다. train/test split은 stratify=y, test_size=0.2, random_state=42로 고정했다. imputation, IQR clipping, scaling, one-hot encoding, feature selection처럼 데이터로부터 학습되는 모든 변환은 train split 이후의 sklearn Pipeline 내부 또는 cross-validation fold 내부에서만 fit되도록 구성했다.

전처리 결정을 위한 EDA 요약

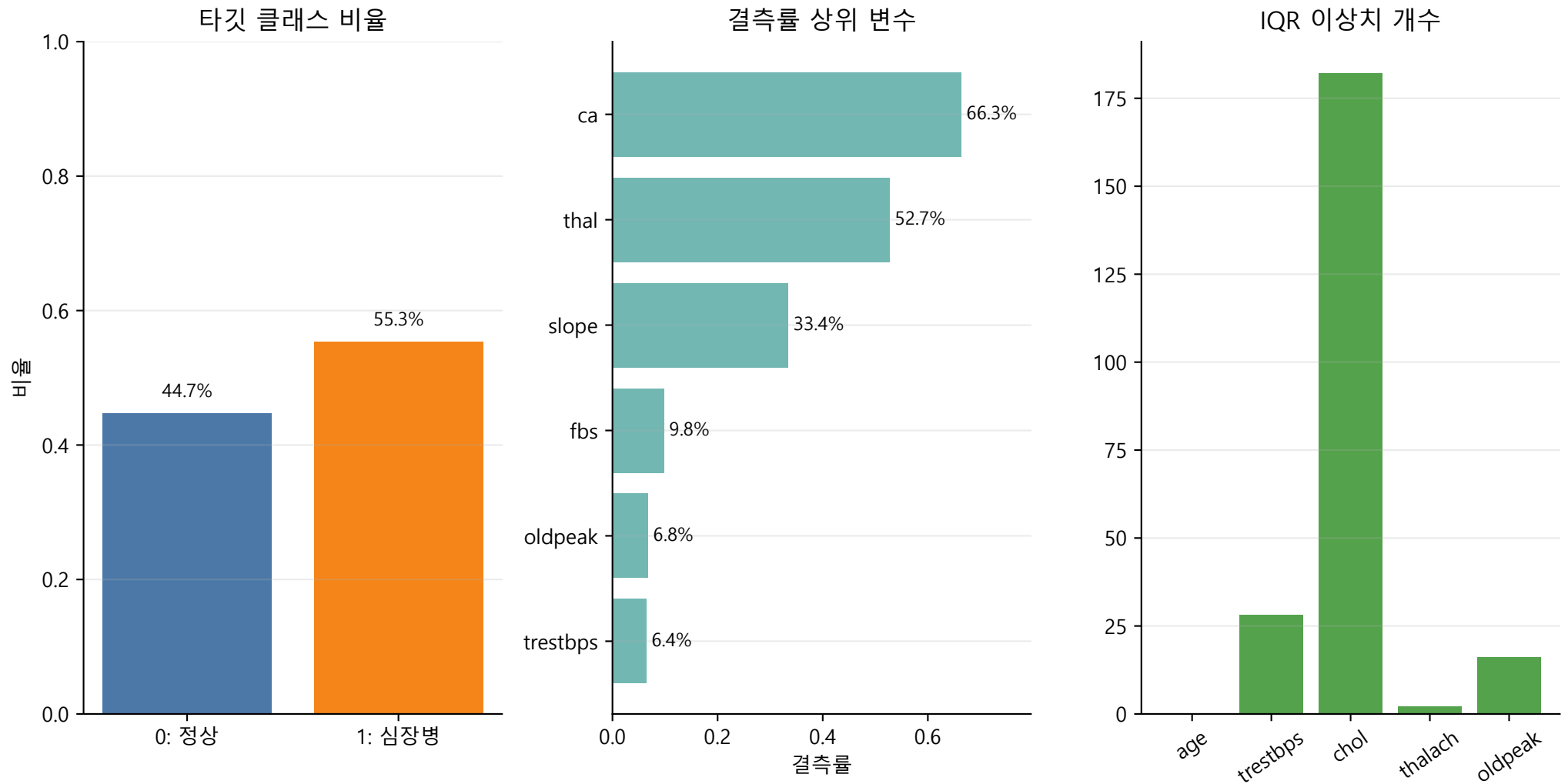


그림 1. 통합 데이터는 약한 클래스 불균형, 수집 기관 차이에 따른 결측, 연속형 변수의 이상치를 함께 보인다. 이 결과를 근거로 stratified split, balanced accuracy/recall 중심 평가, 대치, train-fold IQR clipping, scaling을 적용했다.

EDA 핵심 결과와 전처리 결정

EDA notebook에서는 head, info, describe, target 분포, 결측률, 중복, 임상 범위 점검, 그리고 age, trestbps, chol, thalach, oldpeak의 IQR 이상치를 확인했다. target은 아주 심한 불균형은 아니지만 한쪽으로 기울어져 있어 단순 accuracy만 보면 임상적으로 중요한 오류를 놓칠 수 있다. 그래서 모든 후보 모델에 대해 balanced accuracy, precision, recall, F1, confusion matrix를 함께 보고했다.

통합 UCI 파일에는 여러 임상 변수의 결측이 남아 있다. 네 기관이 동일한 원 schema의 모든 변수를 기록하지 않았기 때문이다. 이 결측 열이나 행을 단순 삭제하면 통합 데이터의 장점을 잃게 되므로, 연속형 변수에는 median imputation을, 코드형 범주 변수에는 most-frequent imputation을 적용했다.

연속형 변수는 scaling 전에 train-fold에서 계산한 IQR 범위로 clipping한다. 이렇게 하면 극단값이 StandardScaler의 평균과 표준편차를 과도하게 흔드는 것을 줄이면서도 새 입력을 동일한 pipeline으로 처리할 수 있다. 코드형 범주 변수는 handle_unknown='ignore'를 둔 one-hot encoding을 사용해, 학습 fold에 없던 category가 추론 시 들어와도 파이프라인이 깨지지 않게 했다. 중복 행과 target이 빈 행은 split 전에 제거했다.

MLflow 실험 기반 모델 비교

run	계열	특성수	CV bAcc	CV prec	CV recall	CV F1	test bAcc	test prec	test recall	test F1
base_logreg	logreg	14	0.807	0.838	0.813	0.825	0.830	0.851	0.843	0.847
base_svc	svc	14	0.827	0.854	0.838	0.845	0.839	0.848	0.873	0.860
base_rf	rf	14	0.803	0.820	0.840	0.829	0.808	0.809	0.873	0.840
tuned_svc	svc	14	0.827	0.854	0.838	0.845	0.839	0.848	0.873	0.860

표 1. 각 run은 parameter, 5-fold CV metric, test metric, 선택 특성, confusion matrix, 학습된 model artifact를 MLflow에 기록한다. 최종 모델은 최고 test balanced accuracy와 0.03 이내인 후보 중 recall, F1, balanced accuracy 순으로 선택했다.

최종 모델 선택 근거

선택된 run: baseline_svc

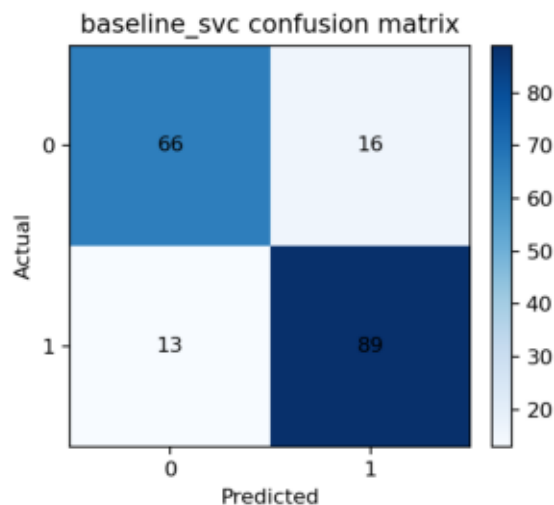
모델 계열: svc

테스트 balanced accuracy: 0.839

테스트 recall: 0.873

테스트 precision: 0.848

테스트 F1: 0.860



최종 모델은 먼저 최고 test balanced accuracy와 0.03 이내인 후보를 남기고, 그 안에서 recall, F1, balanced accuracy 순으로 고르는 규칙을 따랐다. 심장병 가능성을 놓치는 false negative가 추후 진료 지연으로 이어질 수 있기 때문에 recall을 우선순위에 둔 것이다.

Confusion matrix 기준으로 실제 심장병 test record 중 13건은 놓쳤고, 89건은 올바르게 위험군으로 표시했다. 동시에 precision도 보고해 false positive로 인한 추가 검토 부담을 숨기지 않았다. 이 모델은 교육용 의사결정 보조 도구이며, 점수는 차트를 다시 볼 이유이지 의사의 판단을 대체할 이유가 아니다.

그림 2. 최종 모델의 hold-out test confusion matrix.

테스트, 패키징, CI, Repo 구성

unittest는 작지만 실제로 깨질 수 있는 지점을 겨냥했다. 예측 결과의 row 수가 입력 row 수와 일치하는지, predict_proba 값이 [0, 1] 범위에 있고 행별 합이 1인지, cholesterol이 허용 임상 범위를 벗어나면 거부되는지, 같은 seed의 두 pipeline이 같은 입력에 대해 같은 예측을 내는지를 확인한다.

Dockerfile은 python:3.10-slim을 기반으로 pinned requirements를 설치하고, source, data, artifacts, final model을 복사한 뒤 data/sample_input.csv에 대한 batch inference를 실행한다. GitHub Actions workflow는 push와 pull request마다 의존성을 설치하고 python -m unittest를 실행한다.

Repo에는 과제 체크리스트의 핵심 파일을 모두 포함했다: data, notebooks/01_eda_preprocessing.ipynb, src/preprocessing.py, src/train.py, src/inference.py, src/monitor.py, tests/test_pipeline.py, mlruns, Dockerfile, requirements.txt, .github/workflows/ci.yml, report.pdf, README.md. README는 설치, 학습, 추론, 테스트, 모니터링, 보고서 재생성, Docker build 순서를 한 번에 따라갈 수 있게 정리했다.

feature store 후보로는 최대 심박수 변수 thalach가 적절하다. 학습, 추론, drift 점검에 반복적으로 쓰이므로 schema와 freshness 검증 이점이 크다. model registry에는 selected_features를 남기고 싶다. 어떤 변환 특성이 SelectFromModel 이후 살아남았는지 보여 주기 때문에, 모델 버전 비교와 감사 가능성을 높인다.

KS 검정 기반 데이터 드리프트

feature	ks_statistic	p_value	drift_flag
age	0.047	0.879	False
trestbps	0.025	1.000	False
chol	0.381	0.000	True
thalach	0.058	0.706	False
oldpeak	0.908	0.000	True

표 2. monitor.py는 test set 복사본에서 cholesterol과 oldpeak를 인위적으로 이동시킨 뒤, 각 연속형 변수에 scipy.stats.ks_2samp를 적용하고 $p < 0.05$ 를 drift로 표시한다. 이동시키지 않은 변수들이 flag되지 않는 것도 sanity check 역할을 한다.

원본 테스트셋과 드리프트 테스트셋 성능 비교

dataset	balanced_accuracy
original_test	0.839
drifted_test	0.628

표 3. 합성 input drift가 곧바로 모델의 임상적 무효를 의미하지는 않는다. 다만 balanced accuracy 하락은 계속 사용하기 전 재검토가 필요하다는 명확한 신호다.

시간에 따른 모니터링 지표

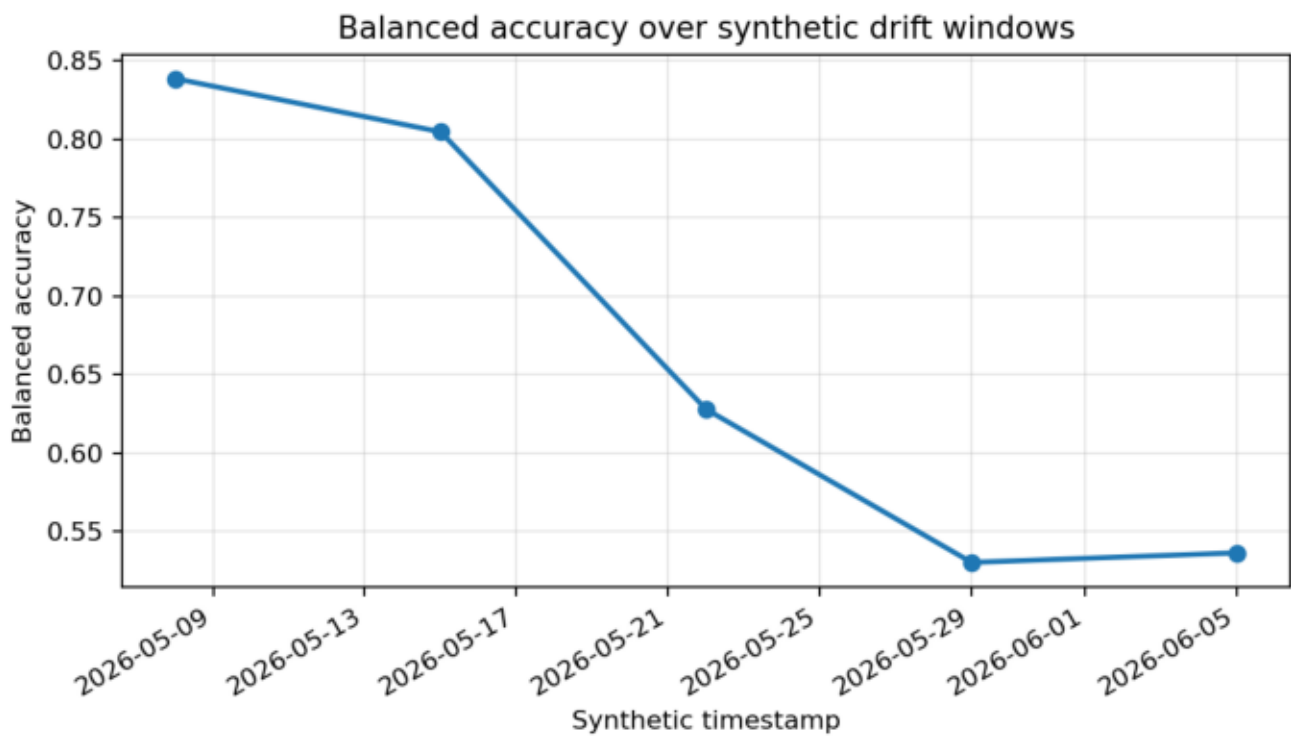


그림 3. 합성 timestamp에 따라 cholesterol shift를 키웠을 때 balanced accuracy가 어떻게 변하는지 보여 준다. 운영 환경에서는 같은 방식으로 model version과 data window별 성능을 추적한다.

서빙, 재학습, 윤리, AI 사용 공개

이 프로젝트는 on-device보다 Model-as-a-Service 형태로 서빙하는 편이 적절하다고 판단했다.

MaaS는 모델 업데이트, audit logging, monitoring, rollback을 단순하게 만든다.

이 과제 수준에서는 작은 latency 이점보다 버전 관리와 감시 가능성이 더 중요하다. PHI는 전송 암호화, 엄격한 접근 제어, 최소 보관, 로그 redaction으로 보호해야 한다. On-device는 데이터 이동을 줄일 수 있지만, 모니터링과 긴급 업데이트가 어려워진다.

재학습은 확인된 drift와 사람이 검토한 label이 함께 있을 때, labeled follow-up data에서 성능 저하가 지속될 때, 또는 정기 임상 review 주기에 맞춰 수행한다.

Human-in-the-loop 검토는 새 label이 training data에 들어가기 전과 새 model version을 올리기 전에 필요하다. 모델이 영향을 준 진료 결정이 그대로 미래 label에 섞이면 runaway feedback loop가 생길 수 있기 때문이다.

가장 큰 한계는 데이터셋이다. UCI Heart Disease는 작고 오래되었으며 현대의 다양한 환자 집단을 충분히 대표하지 않는다. 시간이 한 주 더 있었다면 calibration 분석, subgroup별 성능 점검, 더 명확한 data card, 그리고 임상의가 이해할 수 있는 위험 요인 설명 화면을 추가했을 것이다.

AI 사용 공개. AI 도구는 boilerplate 구현, debugging, 문서 초안 정리에 사용했다. 최종 코드, 실험 선택, 결과 해석, 제출물에 대한 책임은 본인에게 있다.